

Part01 – Questions & Answers

Q1: What are the benefits of using a generic sorting algorithm over a non-generic one?

A: A generic sorting algorithm (like `SortingAlgorithm<T>`) works with any type that satisfies its constraint (e.g. `Comparable<T>`), instead of being written specifically for one data type. This eliminates code duplication, since only one implementation is needed for ints, doubles, custom objects, and any future type; it preserves type safety at compile time, avoiding the casting or boxing/unboxing that a non-generic, object-based sorting method would require; it gives better performance for value types, since boxing overhead is avoided entirely; and it means the sorting logic only ever needs to be written, tested, and maintained once, reducing the risk of a bug being fixed in one type-specific copy but forgotten in another.

Q2: How do lambda expressions improve the readability and flexibility of sorting methods?

A: Lambda expressions let the comparison logic for a sort be written inline, right where the method is called, using concise syntax like `(a, b) => a.CompareTo(b)` instead of a separate, named comparer method or class defined elsewhere. This improves readability, since the exact sorting criteria is visible immediately alongside the call that uses it, rather than requiring the reader to jump elsewhere in the file to understand how the comparison works. It improves flexibility because the same generic sorting method (like `SortingTwo<T>.Sort`) can be reused with completely different sorting criteria simply by passing a different lambda each time, with no new sorting logic or new comparer class needed.

Q3: Why is it important to use a dynamic comparer function when sorting objects of various data types?

A: A dynamic comparer function, passed in as a delegate or lambda (like `Comparison<T>`), lets the same generic sorting method be reused across many different criteria and many different types, instead of hard-coding one fixed comparison rule directly inside the sorting method itself. This matters because real sorting needs vary a lot in practice — sometimes strings need to be sorted by length rather than alphabetically, sometimes objects need to be ordered by a completely different property depending on the situation at hand. A dynamic comparer keeps the sorting algorithm itself generic and reusable, while the actual ordering rule stays fully customizable and is decided by the caller each time it's needed.

Q4: How does implementing `Comparable<T>` in derived classes enable custom sorting?

A: When a derived class implements `Comparable<T>`, it defines its own `CompareTo` method describing exactly how two instances of that specific type should be ordered. This lets any generic algorithm or built-in method relying on `Comparable<T>` — such as `Array.Sort`, or a custom generic sorting class constrained with `where T : Comparable<T>` — correctly sort a collection of that derived type, using ordering logic tailored specifically to it, even if the base class already implements its own comparison logic for the base type. This gives each class in an inheritance hierarchy the freedom to define its own natural ordering rule appropriate to its own data, while still being fully compatible with the exact same generic sorting infrastructure.

Q5: What is the advantage of using built-in delegates like `Func<T, T, TResult>` in generic programming?

A: Built-in generic delegates like `Func<T, TResult>`, `Func<T, T, bool>`, `Action<T>`, and `Predicate<T>` already come defined by the .NET framework, so developers don't need to declare a brand-new custom delegate type every time they need to pass a piece of logic around as a parameter. This saves boilerplate code, keeps APIs consistent and instantly recognizable to any C# developer (since these types are used everywhere across the framework, from LINQ to event handling), and works naturally alongside generic type parameters, so the same `Func<T, T, bool>` shape can be reused for entirely different types — `Employee`, `int`, `string` — without declaring a separate custom delegate for each one.

Q6: How does the usage of anonymous functions differ from lambda expressions in terms of readability and efficiency?

A: Both anonymous methods (using the `delegate` keyword) and lambda expressions let inline logic be defined without a separate named method, but lambda expressions use noticeably more concise syntax — `(a, b) => a + b`

versus `delegate(int a, int b) { return a + b; }` — which generally makes code easier to read at a glance, especially for short expressions. Lambdas also support expression-bodied syntax usable directly in LINQ queries and expression trees, something anonymous methods cannot do. In terms of raw runtime efficiency, both compile down to very similar underlying constructs (a compiler-generated method plus a delegate instance), so there's no meaningful performance difference between them at execution time — the real difference is almost entirely about which syntax reads more clearly, and lambdas are generally preferred in modern C# for that reason.

Q7: Why is the use of generic methods beneficial when creating utility functions like Swap?

A: A generic method like `Swap<T>(ref T a, ref T b)` can operate on any data type — integers, strings, custom structs or classes — using exactly one implementation, instead of requiring a separate, nearly identical method written for every single type that might ever need swapping. This avoids code duplication, reduces the surface area for bugs since there's only one implementation to get right and maintain, and makes the utility immediately usable with any future type introduced into the program, without ever needing to revisit or extend the utility method itself.

Q8: What are the challenges and benefits of implementing multi-criteria sorting logic in generic methods?

A: The main benefit of multi-criteria sorting — such as sorting `Employees` first by `Salary`, and by `Name` only when `Salaries` tie — inside a generic method is that the sorting algorithm itself stays completely unaware of the specific criteria being used; all of that logic lives entirely inside the comparer delegate or lambda passed in by the caller, so the same generic `Sort` method can express arbitrarily complex, multi-level sorting rules without any special-casing inside the algorithm itself. The main challenge is that the comparer logic can get harder to read and more error-prone to write correctly as more criteria and tie-breaking rules are added — especially remembering to only fall through to the next criterion when the current one produces a genuine tie (`CompareTo` returning exactly 0), and keeping the order of criteria (primary, secondary, tertiary...) clear and correct within a single lambda or comparer method.

Q9: Why is the `default(T)` keyword crucial in generic programming, and how does it handle value and reference types differently?

A: Inside a generic method or class, the compiler doesn't know at compile time whether `T` will end up being a value type or a reference type, so it can't simply write 0 or null directly — neither would be valid for every possible `T`. The `default(T)` expression solves this by always producing the correct, safe default value for whatever `T` actually turns out to be at the point it's used: for reference types, `default(T)` is null; for numeric value types, it's the type's zero-equivalent (0, 0.0, and so on); for `bool`, it's false; and for a custom struct, it's that struct with every one of its own fields set to their own default values. This makes `default(T)` essential whenever generic code needs a safe placeholder or initial value for a variable of type `T` without knowing in advance what concrete type it will actually be.

Q10: How do constraints in generic programming ensure type safety and improve the reliability of generic methods?

A: A generic constraint, written with the `where` clause, restricts which types are allowed to be used as `T` for a given generic method or class, ensuring that only types offering the specific capabilities the generic code actually relies on can be plugged in. For example, constraining `T` with `where T : IComparable<T>` guarantees, at compile time, that any type used will genuinely have a working `CompareTo` method, so generic sorting code can safely call `array[j].CompareTo(array[j + 1])` with no risk of that call failing at runtime — the compiler simply won't allow a type lacking that capability to be used there in the first place. This shifts an entire category of potential runtime errors into compile-time errors instead, catching problems immediately during development rather than letting an unsupported type compile successfully and only fail once the program is actually running.

Q11: What are the benefits of using delegates for string transformations in a functional programming style?

A: Using a delegate — a custom one, or a built-in `Func<string, string>` — to represent a string transformation lets the actual transformation logic be passed around as data: a value that can be stored in a variable, passed as a parameter, and swapped out freely, rather than being hard-coded inside the method that applies it. This is the core idea behind functional programming style: the same generic 'apply this transformation to every element' method can perform completely different transformations — uppercase, reverse, trim, encode — just by receiving a different delegate each time, without ever needing to be rewritten or duplicated for each new transformation that comes up.

Q12: How does the use of delegates promote code reusability and flexibility in implementing mathematical operations?

A: A single method like `Calculate(int a, int b, MathOperation operation)` can perform addition, subtraction, multiplication, division, or any other binary operation on two integers, simply by receiving a different delegate each time it's called — the method itself never needs to know or care which specific operation it's actually performing. This means the exact same piece of surrounding logic (whatever validation, logging, or result-handling exists around the calculation) is written once and reused for every possible operation, and adding a brand-new operation later requires no changes at all to the `Calculate` method itself — only a new delegate instance needs to be supplied at the call site.

Q13: What are the advantages of using generic delegates in transforming data structures?

A: A generic delegate — a custom `Transformer<T, R>`, or the built-in `Func<T, TResult>` — lets one single transformation-applying method work for any input type `T` being converted into any output type `R`, instead of requiring a separate transformation method for every possible `T`-to-`R` combination (int-to-string, Employee-to-string, double-to-int...). This makes the transformation infrastructure highly reusable across completely different data structures and completely different kinds of conversions, with the exact conversion logic decided freely by whatever delegate is supplied at the call site, while the core 'loop through the list and transform each element' logic is written and maintained in exactly one place.

Q14: How does Func simplify the creation and usage of delegates in C#?

A: Before generic delegates like `Func<T, TResult>` existed in the .NET framework, using a delegate as a method parameter required first declaring a brand-new, custom delegate type matching the exact signature needed, such as `public delegate int MathOperation(int a, int b)`. `Func<T1, T2, ..., TResult>`, along with its overloads for different numbers of parameters, removes this step entirely for the vast majority of everyday cases, since it already exists as a ready-made, generic delegate type able to represent almost any method signature that returns a value — `Func<int, int>` for one int in, one int out, `Func<int, int, int>` for two ints in, one int out, and so on — eliminating the need to declare a new custom delegate type for every different method shape a program happens to need.

Q15: Why is Action preferred for operations that do not return values?

A: `Action<T>`, along with its parameterless and multi-parameter overloads, is specifically the built-in generic delegate type meant for methods that perform some operation but don't return any value — exactly what `Func` cannot represent cleanly, since every `Func` overload always requires a `TResult` return type as its final type parameter. Using `Action` instead of, say, `Func<T, object>` and simply ignoring or always returning null communicates the intent clearly and correctly at the type level: this delegate exists purely to perform a side effect (printing something, modifying some external state, sending a message), not to compute and hand back a result, which makes calling code and its purpose immediately clear to anyone reading it.

Q16: What role do predicates play in functional programming, and how do they enhance code clarity?

A: A predicate, represented in C# by `Predicate<T>` or more generally any `Func<T, bool>`, is a delegate that takes a single value and returns true or false, expressing a yes/no condition about that value. In a functional programming style, predicates let filtering logic be expressed as reusable, passable data rather than as hard-coded if-statements baked directly into a specific loop — the same generic `FilterList` method can filter for even numbers, filter for strings starting with a certain letter, or filter for virtually any other condition, simply by receiving a different predicate each time. This enhances code clarity because the condition being tested is named and isolated clearly, like `isEven` or `startsWithA`, making the intent of a filtering operation immediately obvious at the call site instead of buried inside a generic loop's body.

Q17: How do anonymous functions improve code modularity and customization?

A: An anonymous function lets a small, specific piece of logic be defined right at the exact point it's needed, as an argument to another method, without requiring a separate, fully named method declared elsewhere in the class purely to hold that one-off piece of logic. This improves modularity because the method receiving the anonymous function — like a generic filter or transform function — stays completely general-purpose and reusable, while each call site supplies its own specific customization inline, without cluttering the rest of the class with many small, single-use named methods that only exist to be passed around once.

Q18: When should you prefer anonymous functions over named methods in implementing mathematical operations?

A: Anonymous functions (or lambda expressions, their more modern equivalent) are generally preferable over separately named methods when the logic involved is short, self-contained, used in only one specific place, and doesn't benefit from having a discoverable name of its own elsewhere in the codebase — for example, a one-line addition or subtraction operation passed directly into a generic Calculate method. A named method becomes the better choice once the logic is more complex, needs to be reused in multiple different places, benefits from having a clear, descriptive name that documents its purpose, or needs to be unit-tested independently as its own distinct unit of code.

Q19: What makes lambda expressions an essential feature in modern C# programming?

A: Lambda expressions provide a concise, readable way to write inline functions, which makes them foundational to a huge amount of modern C# code: they are what LINQ queries are built on almost entirely (Where(x => x > 5), Select(x => x.Name), and so on), they're the standard way to supply short custom logic to built-in delegate types like Func, Action, and Predicate, they support async lambdas for asynchronous programming, and they can be compiled into expression trees — which is exactly what lets technologies like Entity Framework translate C# code directly into SQL queries. Without lambda expressions, a huge portion of everyday modern C# code — LINQ, event handling, async workflows, functional-style data transformations — would require noticeably more verbose, separately-declared named methods for even the simplest pieces of logic.

Q20: How do lambda expressions enhance the expressiveness of mathematical computations in C#?

A: Lambda expressions let a mathematical operation — division, exponentiation, or any custom formula — be written and passed around as a compact, self-contained expression, like (a, b) => Math.Pow(a, b), directly at the point it's needed, instead of requiring a full, separately declared method just to express a single calculation. This makes code that works with configurable or swappable mathematical operations noticeably more expressive and readable, since the actual formula being used is visible immediately, right where it's supplied, rather than being hidden away inside a differently-named method the reader would have to go look up separately just to understand what calculation is actually being performed.

Part02 – Questions & Answers

Q2: Search about these topics (Parallel Programming and Concurrency - Unit Testing and Test-Driven Development (TDD) - Asynchronous Programming with async and await)

A:

Parallel Programming and Concurrency: parallel programming means running multiple pieces of work literally at the same time, typically across multiple CPU cores, to make CPU-heavy work finish faster (in C#, tools like Parallel.For, Parallel.ForEach, and the Task Parallel Library support this). Concurrency is the broader idea of managing multiple tasks that are logically in progress 'at once', even if they aren't literally executing simultaneously — it's about structuring a program so multiple operations can make progress without blocking each other, which introduces its own challenges like race conditions (multiple threads unsafely accessing shared data at the same time) and the need for synchronization tools like locks.

Unit Testing and Test-Driven Development (TDD): unit testing means writing small, automated tests that each verify one specific piece of behavior in isolation — usually a single method or a small unit of logic — so changes to the code can be verified quickly and confidently without manually re-checking everything by hand. In C#, this is commonly done with frameworks like MSTest, NUnit, or xUnit. TDD is a development practice built around writing the test BEFORE writing the actual implementation: first write a test describing the wanted behavior (which initially fails, since the feature doesn't exist yet), then write just enough code to make it pass, then refactor while keeping the test passing — repeating this 'red, green, refactor' cycle for each new piece of functionality.

Asynchronous Programming with async and await: this lets a program start a long-running operation — a file read, a network call, a database query — without blocking the thread that started it, keeping the application responsive (especially in UI apps) or free to handle other work (especially in server apps) while that operation is still in progress. The async keyword marks a method as asynchronous, and await pauses that method's execution at that point, without blocking the underlying thread, until the awaited operation completes, after which execution resumes from where it left off.

Part03 Bonus – Question

Self study

C# / OOP — Self Study

1. is Conditional Operator & as Cast Operator

is Operator

The is operator is used to check whether an object is compatible with a specific type.

It returns a Boolean result:

- True → the object is compatible with the specified type.
- False → the object is not compatible with the specified type.

In modern C#, is can also be used with pattern matching, allowing type checking and obtaining a strongly typed reference after the check succeeds.

as Operator

The as operator is used to attempt a type conversion.

If the conversion succeeds, it returns the object as the requested type.

If the conversion fails for a reference type, it returns null instead of throwing an exception.

Difference Between is and as

is:

- Mainly used for type checking.
- Returns true or false.
- Supports pattern matching.

as:

- Mainly used for attempting a type conversion.
- Returns the converted reference or null.
- Useful when the converted object will be used afterward.

Main Idea

is → "Is this object of this type?"

as → "Try to convert this object to this type."

2. C# 7.0 — Enum

An Enum (Enumeration) is a value type used to represent a fixed set of named constants.

Instead of using unclear numbers or strings to represent a specific state, an Enum provides meaningful names.

Common use cases include:

- Order Status

- User Roles
- Days
- Payment Status
- Account Status

Enums improve:

- Readability
- Type Safety
- Maintainability

By default, each Enum member has an underlying integer value, normally starting from 0 and increasing sequentially. Custom values can also be assigned.

Main Idea

An Enum is useful when you have a limited and predefined set of related choices.

3. Constructor — One Use Case

A Constructor is a special member of a class that is automatically executed when an object is created.

One of its main purposes is initializing the object and establishing its initial valid state.

Instead of creating an object and then setting its required information separately, a constructor can require the necessary data when the object is created.

Use Case

For example, an Employee may require basic information such as:

- ID
- Name
- Department

A constructor can ensure that the Employee object starts with the required information.

Main Idea

Constructor → Initialize the object and establish its initial valid state.

4. Custom Delegate — Use Case

A Delegate is a type-safe reference to a method.

It allows a method to be treated as a value and passed to another method.

A Custom Delegate is useful when we want to define the expected signature of methods that can be passed and executed.

Use Case

Imagine a notification system that can send notifications through:

- Email
- SMS
- Push Notification

Instead of making the main process depend on one specific notification method, a delegate can allow the required behavior to be passed into the process.

Benefits

Delegates provide:

- Flexibility
- Loose Coupling
- Passing behavior
- Callback mechanisms

Main Idea

Delegate → Allows a method to be passed as a parameter and executed when needed.

5. Code Performance — Alternative Approach

When focusing on performance, we should not assume that one implementation is faster simply because it looks simpler.

We should consider different approaches and compare them based on:

- Time Complexity
- Space Complexity
- Data Structures
- Number of operations
- Data size
- Actual workload

Example Concept

Suppose we have a large collection of Employees and frequently need to find an Employee using an ID.

Searching through a List usually requires checking elements sequentially, so the search is generally $O(n)$.

A Dictionary, however, is designed for lookup using a key, and its average lookup complexity is $O(1)$.

Therefore, choosing the correct data structure can have a significant impact on performance.

Main Idea

Performance optimization is not about writing less code.

It is about:

Choosing the right algorithm and data structure for the problem.

Optimization should also be based on actual measurements and profiling when performance becomes a real concern.

6. Record

A Record is a C# type designed mainly for representing data, with a focus on value-based equality.

The main difference between a record and a traditional class is how equality is handled.

With a normal class, equality is generally based on object identity/reference unless the equality behavior is explicitly changed.

With a record, equality is based on the values represented by the object.

Main Benefits of Records

Records are useful for:

- Representing data
- DTOs
- API responses
- Configuration objects
- Immutable or mostly immutable data
- Situations where value-based equality is important

They also reduce the amount of boilerplate code needed for data-oriented types.

Main Idea

Class → Usually represents an object/entity and its behavior.

Record → Usually represents data/value.

7. Readable Code — Type Inference — Expression Tree

Readable Code

Readability means that code is easy for developers to understand, maintain, and modify.

Readable code is not necessarily the shortest code.

Sometimes explicitly writing information makes code clearer, while in other situations removing unnecessary repetition makes it cleaner.

The goal is to find a balance between:

- Simplicity
 - Clarity
 - Maintainability
-

Type Inference

Type Inference is the compiler's ability to determine the type of a variable automatically based on its assigned value or expression.

It helps reduce unnecessary repetition and can make code cleaner.

However, Type Inference does not mean that C# becomes dynamically typed.

When using var, the compiler still determines the actual type at compile time.

Main Idea

Type Inference → The compiler determines the type automatically.

Expression Tree

An Expression Tree represents an expression as a data structure that can be inspected, analyzed, and potentially translated.

This is especially important with frameworks such as Entity Framework.

When using LINQ with Entity Framework, the framework can inspect the structure of an expression and translate it into SQL that can be executed against the database.

Expression Tree and Database

The general process can be understood as:

C# Expression → Expression Tree → Entity Framework → SQL → Database

This is different from simply executing a normal C# function.

Delegate vs Expression Tree

Delegate:

- Represents executable code.
- Main purpose is to execute behavior.

Expression Tree:

- Represents the structure of code as data.
- Can be inspected and analyzed.
- Can be translated by frameworks such as Entity Framework.

Main Idea

Expression Tree → Code represented as data so it can be analyzed or translated.

Q: What is Asynchronous programming?

A: Asynchronous programming is a programming model that allows an operation which takes some time to complete — reading a file, calling a web API, querying a database — to run without blocking the thread that started it, so that thread stays free to do other work, or in a UI application, keeps the interface responsive, while the operation is still in progress. In C#, this is primarily implemented through the `async` and `await` keywords together with the `Task` and `Task<TResult>` types: a method marked `async` can use `await` on a `Task`, which pauses that specific method's execution at that point — without blocking the actual underlying thread — until the awaited task completes, after which the rest of the method's code resumes running. This is fundamentally different from parallel programming, which is about doing multiple things literally simultaneously across multiple CPU cores to speed up CPU-heavy work; asynchronous programming instead is mainly about not wasting a thread sitting idle and blocked while waiting on an operation that spends most of its time waiting on something external — disk, network, another server — rather than actually using the CPU.